Nombre del juego: DHTowerdefense

Versión 1.0

Autor: Diego Francisco Hernández Ramírez

Motor y versión: Godot Engine 4.2.2

Lenguaje: GDScript

Fecha: 28 de febrero del 2026

Resumen:

DHTowerDefense es un videojuego del género Tower Defense desarrollado con el motor Godot 4.2.2.
El jugador debe defender su territorio colocando diferentes tipos de guerreros estratégicamente en el campo de batalla para detener las oleadas de enemigos.
Cada unidad posee estadísticas y comportamientos únicos, lo que obliga al jugador a tomar decisiones tácticas en tiempo real.

El proyecto fue desarrollado como una primera versión funcional enfocada en consolidar conocimientos en arquitectura de escenas, señales, control de estados del juego y manejo de colisiones.

Concepto del juego:

El objetivo principal del juego es sobrevivir a múltiples oleadas de enemigos utilizando distintos tipos de guerreros.

El jugador dispone de recursos limitados (moneda “Gloria”) para colocar unidades estratégicamente en celdas del mapa.

La dificultad aumenta progresivamente a lo largo de 5 niveles, culminando en una pantalla final de campaña completada.

Inspiración:

El diseño del juego toma como referencia el modelo estratégico introducido por Plants vs. Zombies, particularmente en la organización por filas y el uso de unidades estacionarias con ataque automático, El juego toma como referencia el modelo estratégico popularizado por Plants vs. Zombies, especialmente en la organización por filas y el sistema de unidades estacionarias con ataque automático.
Sin embargo, se desarrolló una versión propia con identidad visual y estructura modular personalizada.

Mecánicas base

Gameplay / Mecánicas:

Los enemigos aparecen organizados en filas y avanzan automáticamente hacia el extremo del mapa.

Si un enemigo alcanza el final del recorrido, el jugador pierde.

Sistema de combate

Las unidades atacan automáticamente cuando detectan enemigos en su misma fila.

El ataque se realiza mediante la generación de proyectiles (Area2D), los cuales detectan colisiones con los enemigos y aplican daño.

Sistema de proyectiles

Cada guerrero genera su propio tipo de proyectil:

- Arquero → Flecha
- Lancero → Lanza
- Guerrero Hacha → Hacha

Cada proyectil es una escena independiente con su propia animación de impacto, evitando conflictos visuales entre unidades.

Sistema de daño

El daño se aplica mediante el método *recibir_ataque()* dentro de la clase Enemigo.

Cada unidad posee una variable exportada que define el daño por disparo.

Ejemplo conceptual:

enemigo.recibir_ataque(daño)

Sistema de moneda (gloria)

El jugador recibe moneda al eliminar enemigos.

La moneda se utiliza para colocar nuevas unidades en el campo.

Sistema de pausa

El juego puede pausarse mediante una interfaz gráfica o presionando la tecla ESC.

Se implementó utilizando *get_tree().paused = true* y configurando nodos críticos con *process_mode = PROCESS_MODE_ALWAYS.*

Sistema de victoria y derrota

Existen tres estados finales:

1. Victoria por nivel
2. Derrota (Game Over)
3. Victoria final de campaña

Cada uno se maneja con una escena independiente tipo *Control*.

Personajes / Unidades

Guerrero Arquero

- Daño: 10
- Tipo de ataque: Proyectil (Flecha)
- Velocidad de ataque: Media

Unidad básica de ataque a distancia. Ideal para oleadas iniciales.

Guerrero Lancero

- Daño: 15
- Tipo de ataque: Proyectil (Lanza)
- Velocidad de ataque: Alta

Versión mejorada del arquero con mayor daño y velocidad de ataque.

Guerrero Hacha

- Daño: 20
- Tipo de ataque: Proyectil (Hacha)
- Especialización: Daño elevado
- Velocidad de ataque: Menor que el arquero

Unidad diseñada para ataques más contundentes.

ENEMIGOS

Los enemigos avanzan automáticamente por su fila asignada.

Cada enemigo posee:

- Vida
- Velocidad
- Método *recibir_ataque()*
- Animación de muerte

Son 3 tipos diferentes enemigos, con mayor vida, daño de ataque y daño especial, atacando a distancia.

Arquitectura del Proyecto:

Organización por escenas

El proyecto está estructurado bajo un enfoque modular:

- Cada unidad es una escena independiente.
- Cada proyectil es una escena independiente.
- Cada impacto es una escena independiente.
- Las pantallas de UI son escenas tipo Control.

Uso de señales

Se utilizaron señales para comunicar eventos entre nodos, como:

- Finalización de animaciones
- Timeout de timers
- Detección de colisiones

Esto permitió desacoplar la lógica del proyecto.

Game Manager

Se implementó un GameManager para:

- Controlar el nivel actual
- Manejar la progresión
- Cargar escenas
- Reiniciar partidas

Esto centraliza el control del flujo del juego.

Diagrama conceptual de clases

Personajes:

GuerreroBase
↑
GuerreroArquero
↑
GuerreroLancero
↑
GuerreroHacha

Exporto las variables de vida máxima, vida actual, daño por proyectil, recibir ataque.

Cada uno hereda vida máxima, vida actual, daño por proyectil y recibir ataque.

Y a cada guerrero le sobrescribo la vida máxima y daño por proyectil.

Enemigos:

Enemigo
↑
HombreLobo
↑

Bruja

Exporto las variables de vida máxima, vida actual, daño por ataque, recibir ataque, velocidad y animaciones de estado, movimiento, ataque y abatido.

Cada enemigo hereda la vida máxima, vida actual junto con el daño, recibir ataque, velocidad y sus animaciones, con excepción de la bruja la cual cuenta con una animación extra, de proyectil e impacto proyectil.

Sobrescribo la vida máxima, el daño y su velocidad, a la bruja le agrego el daño de proyectil aparte de su daño heredado.

Flujo del Juego

Flujo del juego

1. El jugador entra al menú.
2. El jugador entra al nivel.
3. Se inicializa gloria.
4. Se crean celdas.
5. Se inician oleadas.
6. EnemyManager gestiona spawn y manda la señal para que no inicie otra oleada sin antes terminar con todos los enemigos de la oleada actual.
7. Si enemigo llega al final → GameOver.
8. Si oleadas terminan → Victoria y se avanza al siguiente nivel.

Problemas Encontrados y Soluciones

Problema 1 – impacto compartido entre proyectiles

Situación:
Todos los proyectiles utilizaban el mismo archivo de impacto, provocando que la lanza y el hacha mostraran animación de flecha.

Solución:
Se crearon escenas de impacto independientes por tipo de proyectil.

Problema 2 – colisiones sin aplicar daño

Situación:
La detección imprimía colisión, pero no aplicaba daño, del ataque especial a distancia de un enemigo.

Solución:
Se corrigió el uso de *area_entered* y la referencia correcta al nodo padre del enemigo.

Problema 3 – estado persistente al reiniciar campaña

Situación:
Al volver al menú y reiniciar, permanecían guerreros desbloqueados y celdas ocupadas.

Solución:
Se implementó reinicio de variables globales dentro del GameManager.

Estructura de Carpetas

/assets

/MainMenu

Como su nombre lo indica aquí tenemos la interfaz principal del menú y primera pantalla a mostrar.

/Framework

Aquí están los diversos niveles.

/Clases

Aquí tenemos los diversos guerreros, junto con las celdas, matriz del mapa y condiciones de interacción junto con señales para los diversos eventos.

/Enemigos

Aquí tenemos los diversos enemigos para ser llamados al mapa.

/Global

Aquí tenemos las clases globales, *enemy_manager, game_manager y global.* Con las cuales puedo iniciar las diversas oleadas y preparar la siguiente oleada mientras el jugador derrota la oleada actual.

/UI

Aquí tenemos la interfaz con la cual puede interactuar el jugador, ver su panel de guerreros, bloqueados y liberados, el contador de las monedas junto con la barra de progreso de las oleadas, también tenemos las pantallas de pausa, derrota *game over,* victoria por nivel y victoria por finalizar la campaña.

/MusicSound

Como su nombre lo indica aquí tenemos los archivos de audios, música, efectos y sonidos para complementar el juego.

/Objetos

Aquí tenemos los objetos que aparecerán en el mapa, como lo son: monedas, proyectiles, proyectiles impacto y los diversos proyectiles de cada guerrero.

/Guerreros

Aquí tenemos las escenas de cada guerrero para instanciarlas en el panel de la interfaz del usuario.

Decisiones Técnicas

Por qué separaste los proyectiles por tipo:

Decidí separar los proyectiles debido a que usaban la misma escena de impacto y para evitar confusión con el código, debido que los guerreros heredan características del guerrero base y no sabe cuál animación de impacto usar en cada guerrero, al separarlos puedo decirle a cada guerrero que escena y animación usar al instanciar su proyectil.

Por qué cada impacto tiene su escena independiente:

La animación usa el nodo de AnimatedSprite2D, para evitar duplicar código y decirle a cada guerrero que tenga lista sus escenas de proyectil y proyectil impacto todo el tiempo, separe las escenas para cuando ocurra su evento mande a llamar mi escena y reproduzca la animación.

Mejoras Futuras (Roadmap)

- Sistema de habilidades especiales (Colocarle a cada guerrero un ataque especial o mayor número de proyectiles disparados tras derrotar cierta cantidad de enemigos).
- Guardado permanente (Crear una pantalla de los niveles y que se quede guardado tu progreso, para poder jugar donde los dejaste, aunque cierres el juego).
- Más tipos de enemigos (Tengo planeado más tipos de enemigos y un par de jefes finales cada cierta cantidad de niveles).
- Efectos de sonido dinámicos (Mejorar los sonidos del juego para una mejor inmersión)
- Música adaptativa (Implementación de diversos tipos de música para el menú, pantalla de pausa, esperando oleada, completando oleada y victoria por niveles).
- Mejorar los gráficos (Pese ser un juego 2D, se pueden mejorar los gráficos, debido a que no soy un artista, genere todo lo visual usando 2 tipos de ias para las ideas y las animaciones que tiene el juego actual).
- Que los guerreros tengan un cierto rango de ataque y 2 tipos de ataque, por proyectil y cuerpo a cuerpo.
- Mejorar la interfaz del usuario, que sea más agradable visualmente.
- Agregar más niveles y mejorar la sensación de las oleadas.
- Crear diferentes mapas para los niveles.

Requisitos técnicos

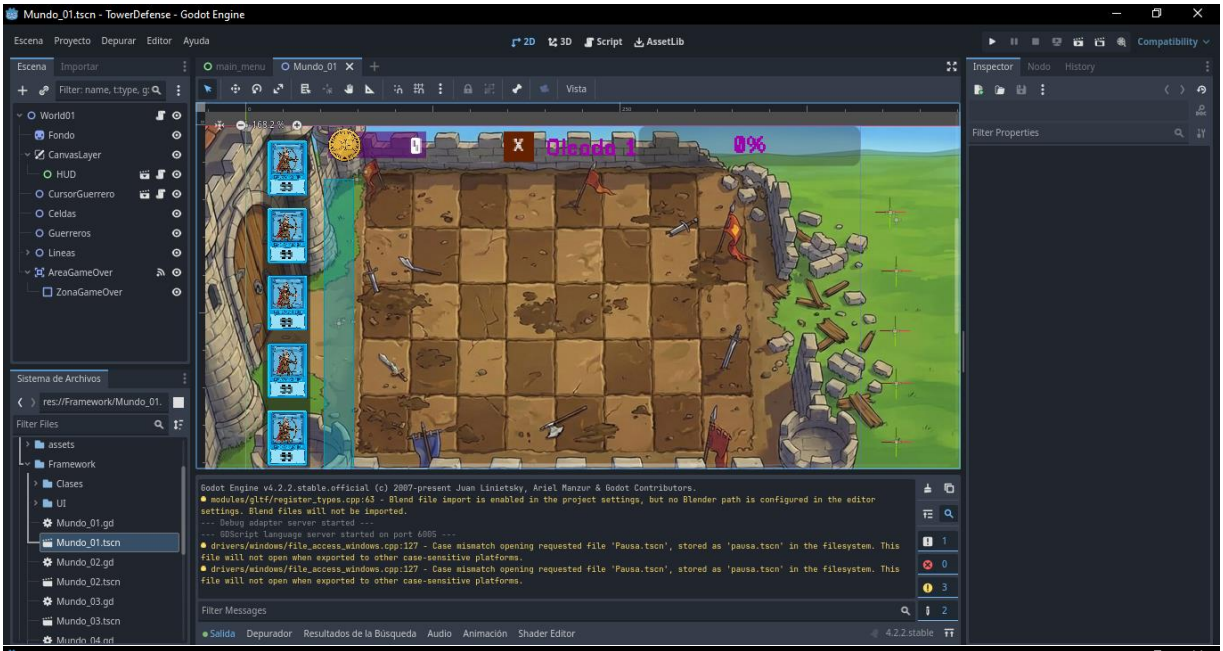
Requisitos

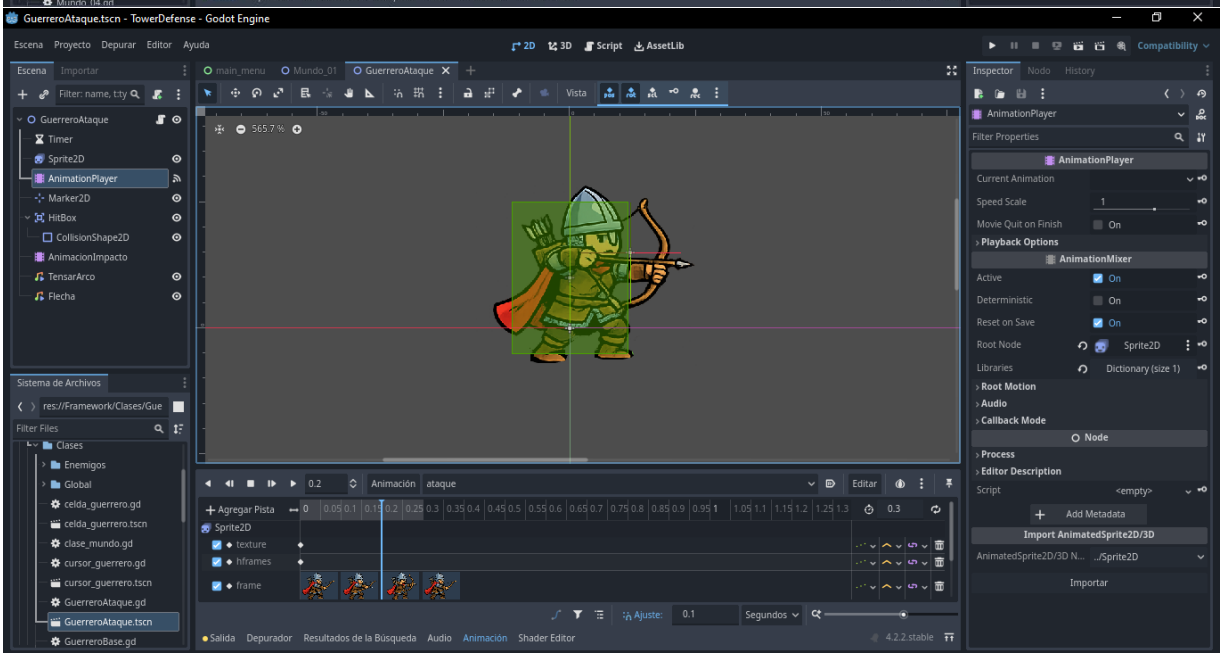
- Sistema Operativo: Windows 10+
- Motor: Godot 4.2.2
- 2 GB RAM
- Gráficos integrados
- Arquitectura: 64 bits

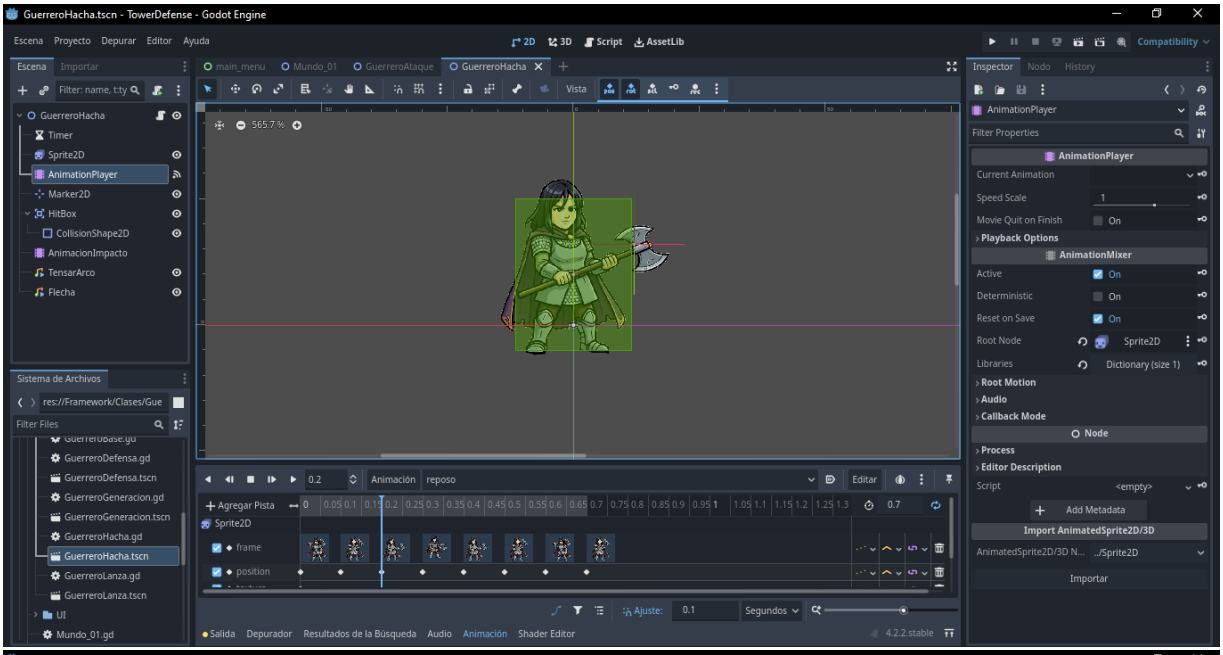
Capturas del juego

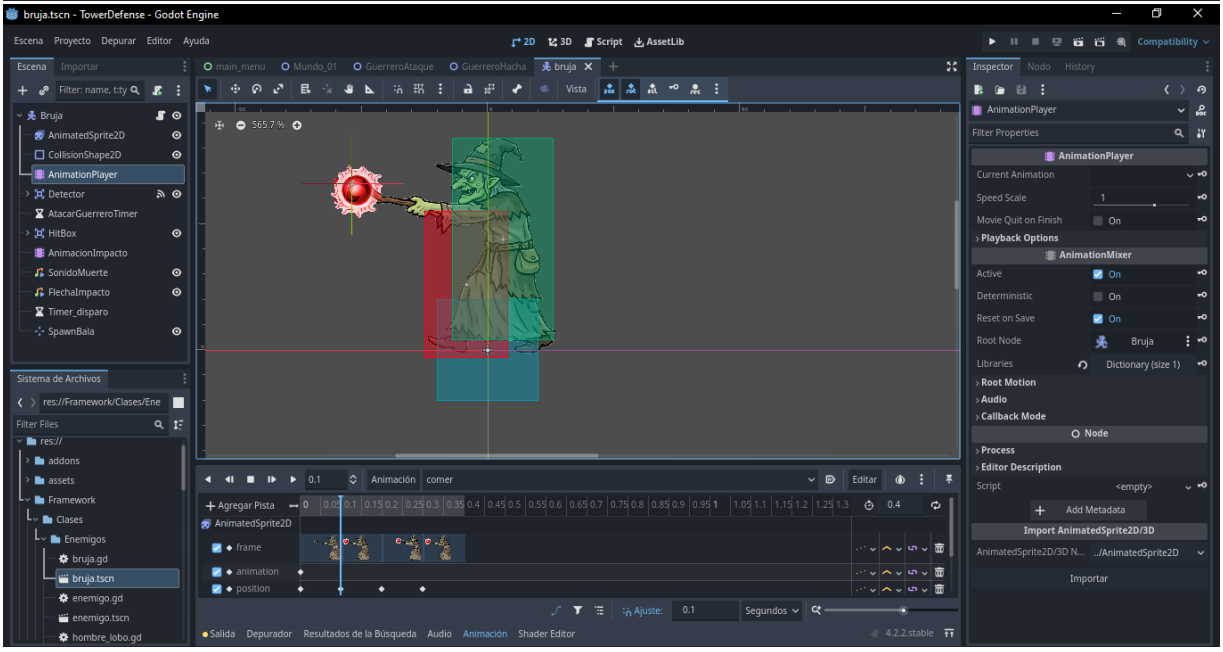
TowerDefense (DEBUG)
300
Oleada 4
100%
50
100
50
125
150
VICTORIA
Siguiente nivel
MENU

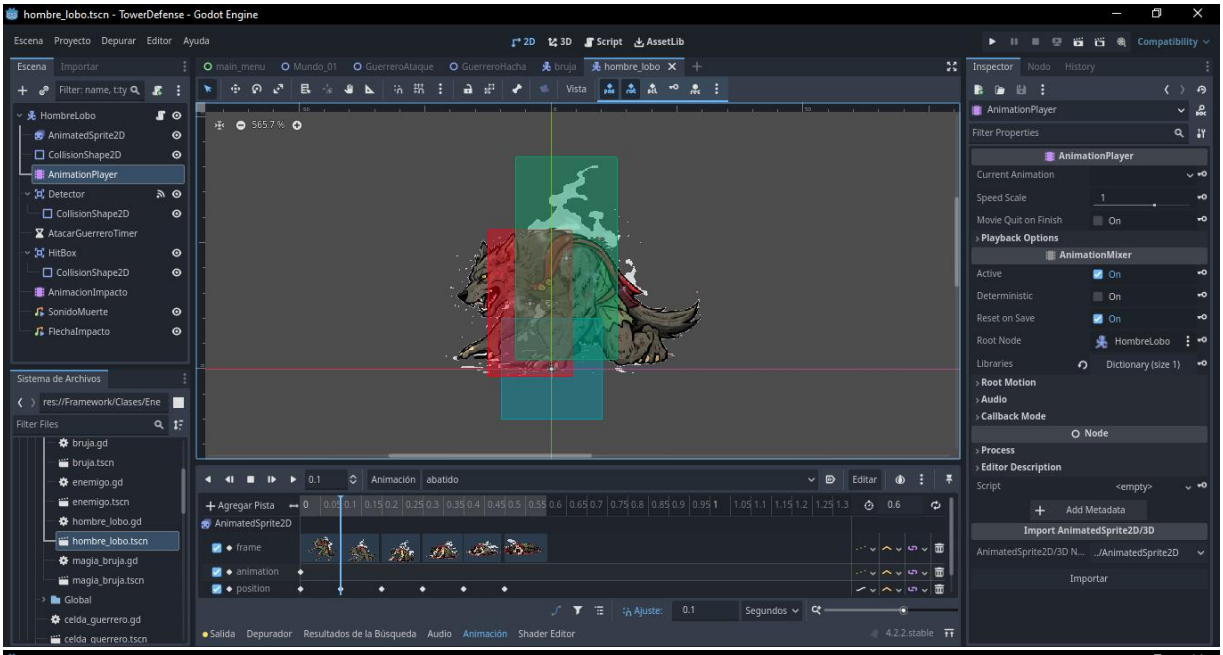
TowerDefense (DEBUG)
PLAY

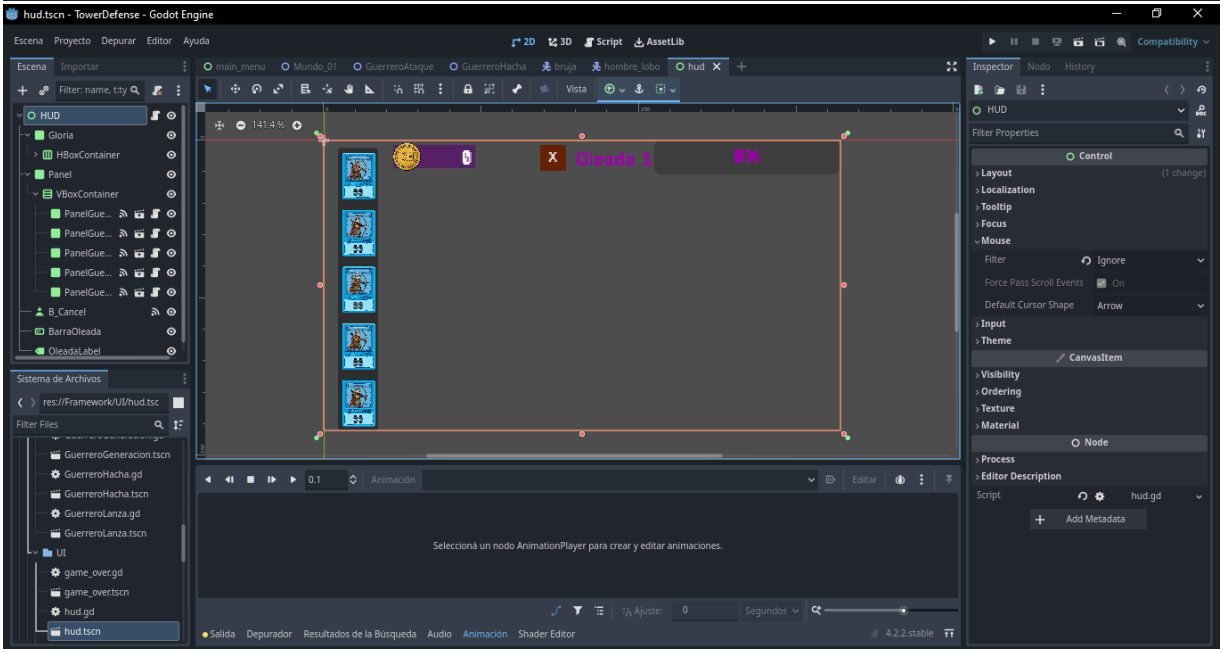

Mundo_01.tscn - TowerDefense - Godot Engine
Escena Proyecto Depurar Editor Ayuda
2D 3D Script AssetLib
Compatibility
Escena Importar
main_menu Mundo_01
Vista
Inspector Nodo History
World01
Fondo
CanvasLayer
HUD
CursorGuerrero
Celdas
Guerreros
Lineas
AreaGameOver
ZonaGameOver
Filter Properties
Sistema de Archivos
Filter Files
assets
Framework
Clases
UI
Mundo_01.gd
Mundo_01.tscn
Mundo_02.gd
Mundo_02.tscn
Mundo_03.gd
Mundo_03.tscn
Filter Messages
Salida Depurador Resultados de la Búsqueda Audio Animación Shader Editor
4.2.2.stable


GuerreroAtaque.tscn - TowerDefense - Godot Engine
Escena Proyecto Depurar Editor Ayuda
2D 3D Script AssetLib
Compatibility
Escena Importar
main_menu Mundo_01 GuerreroAtaque
Vista
Inspector Nodo History
GuerreroAtaque
Timer
Sprite2D
AnimationPlayer
Marker2D
HitBox
CollisionShape2D
AnimacionImpacto
TensarArco
Flecha
AnimationPlayer
Filter Properties
Current Animation
Speed Scale
Movie Quit on Finish
Playback Options
AnimationMixer
Active
Deterministic
Reset on Save
Root Node
Libraries
Root Motion
Audio
Callback Mode
Node
Process
Editor Description
Script
Add Metadata
Import AnimatedSprite2D/3D
Importar
Sistema de Archivos
Filter Files
Clases
Enemigos
Global
celda_guerrero.gd
celda_guerrero.tscn
clase_mundo.gd
cursor_guerrero.gd
cursor_guerrero.tscn
GuerreroAtaque.gd
GuerreroAtaque.tscn
GuerreroBase.gd
Animación ataque
Editar
Agregar Pista
Sprite2D
texture
hframes
frame
Ajuste: 0.1 Segundos
Salida Depurador Resultados de la Búsqueda Audio Animación Shader Editor
4.2.2.stable


GuerreroHacha.tscn - TowerDefense - Godot Engine
Escena Proyecto Depurar Editor Ayuda
2D 3D Script AssetLib
Compatibility
Escena Importar
Filter: name, t:ty
GuerreroHacha
Timer
Sprite2D
AnimationPlayer
Marker2D
HitBox
CollisionShape2D
AnimacionImpacto
TensarArco
Flecha
Sistema de Archivos
res://Framework/Clases/Gue
Filter Files
GuerreroDefensa.gd
GuerreroDefensa.tscn
GuerreroGeneracion.gd
GuerreroGeneracion.tscn
GuerreroHacha.gd
GuerreroHacha.tscn
GuerreroLanza.gd
GuerreroLanza.tscn
UI
Mundo_01.gd
main_menu
Mundo_01
GuerreroAtaque
GuerreroHacha
Vista
565.7 %
Animación
reposo
Editar
Agregar Pista
Sprite2D
frame
position
Ajuste:
Segundos
Salida Depurador Resultados de la Búsqueda Audio Animación Shader Editor
4.2.2.stable
Inspector Nodo History
AnimationPlayer
Filter Properties
Current Animation
Speed Scale
Movie Quit on Finish
Playback Options
AnimationMixer
Active
Deterministic
Reset on Save
Root Node
Libraries
Root Motion
Audio
Callback Mode
Node
Process
Editor Description
Script
Add Metadata
Import AnimatedSprite2D/3D
AnimatedSprite2D/3D N...
../Sprite2D
Importar


bruja.tscn - TowerDefense - Godot Engine
Escena Proyecto Depurar Editor Ayuda
2D 3D Script AssetLib
Compatibility
Escena Importar
Filter: name, t:ty
Bruja
AnimatedSprite2D
CollisionShape2D
AnimationPlayer
Detector
AtacarGuerreroTimer
HitBox
AnimacionImpacto
SonidoMuerte
FlechaImpacto
Timer_disparo
SpawnBala
Sistema de Archivos
res://Framework/Clases/Ene
Filter Files
res://
addons
assets
Framework
Clases
Enemigos
bruja.gd
bruja.tscn
enemigo.gd
enemigo.tscn
hombre_lobo.gd
main_menu
Mundo_01
GuerreroAtaque
GuerreroHacha
bruja
Vista
565.7 %
Animación
comer
Editar
Agregar Pista
AnimatedSprite2D
frame
animation
position
Ajuste:
Segundos
Salida Depurador Resultados de la Búsqueda Audio Animación Shader Editor
4.2.2.stable
Inspector Nodo History
AnimationPlayer
Filter Properties
Current Animation
Speed Scale
Movie Quit on Finish
Playback Options
AnimationMixer
Active
Deterministic
Reset on Save
Root Node
Libraries
Root Motion
Audio
Callback Mode
Node
Process
Editor Description
Script
Add Metadata
Import AnimatedSprite2D/3D
AnimatedSprite2D/3D N...
../AnimatedSprite2D
Importar


hombre_lobo.tscn - TowerDefense - Godot Engine
Escena Proyecto Depurar Editor Ayuda
2D 3D Script AssetLib
Compatibility
Escena Importar
Filter: name, t:ty
HombreLobo
AnimatedSprite2D
CollisionShape2D
AnimationPlayer
Detector
CollisionShape2D
AtacarGuerreroTimer
HitBox
CollisionShape2D
AnimacionImpacto
SonidoMuerte
FlechaImpacto
Sistema de Archivos
res://Framework/Clases/Ene
Filter Files
bruja.gd
bruja.tscn
enemigo.gd
enemigo.tscn
hombre_lobo.gd
hombre_lobo.tscn
magia_bruja.gd
magia_bruja.tscn
Global
celda_guerrero.gd
celda_guerrero.tscn
main_menu
Mundo_01
GuerreroAtaque
GuerreroHacha
bruja
hombre_lobo
Vista
565.7 %
Animación
abatido
Editar
Agregar Pista
AnimatedSprite2D
frame
animation
position
Ajuste:
Segundos
Salida Depurador Resultados de la Búsqueda Audio Animación Shader Editor
4.2.2.stable
Inspector Nodo History
AnimationPlayer
Filter Properties
AnimationPlayer
Current Animation
Speed Scale
Movie Quit on Finish
Playback Options
AnimationMixer
Active
Deterministic
Reset on Save
Root Node
HombreLobo
Libraries
Dictionary (size 1)
Root Motion
Audio
Callback Mode
Node
Process
Editor Description
Script
<empty>
Add Metadata
Import AnimatedSprite2D/3D
AnimatedSprite2D/3D N...
../AnimatedSprite2D
Importar


hud.tscn - TowerDefense - Godot Engine
Escena Proyecto Depurar Editor Ayuda
2D 3D Script AssetLib
Compatibility
Escena Importar
Filter: name, t:ty
HUD
Gloria
HBoxContainer
Panel
VBoxContainer
PanelGue...
PanelGue...
PanelGue...
PanelGue...
PanelGue...
B_Cancel
BarraOleada
OleadaLabel
Sistema de Archivos
res://Framework/UI/hud.tsc
Filter Files
GuerreroGeneracion.tscn
GuerreroHacha.gd
GuerreroHacha.tscn
GuerreroLanza.gd
GuerreroLanza.tscn
UI
game_over.gd
game_over.tscn
hud.gd
hud.tscn
main_menu
Mundo_01
GuerreroAtaque
GuerreroHacha
bruja
hombre_lobo
hud
Vista
141.4 %
X
Oleada 1
0%
Animación
Editar
Seleccioná un nodo AnimationPlayer para crear y editar animaciones.
Ajuste:
Segundos
Salida Depurador Resultados de la Búsqueda Audio Animación Shader Editor
4.2.2.stable
Inspector Nodo History
HUD
Filter Properties
Control
Layout
Localization
Tooltip
Focus
Mouse
Filter
Ignore
Force Pass Scroll Events
Default Cursor Shape
Arrow
Input
Theme
CanvasItem
Visibility
Ordering
Texture
Material
Node
Process
Editor Description
Script
hud.gd
Add Metadata

Conclusión

El desarrollo de DHTowerDefense representó una consolidación de conocimientos en programación orientada a nodos dentro de Godot.

Se aplicaron conceptos como herencia, señales, modularización de escenas y control de estados globales.

Este proyecto sienta las bases para futuras mejoras y una versión expandida con mayor complejidad mecánica.

Aprendizajes del Proyecto

- Arquitectura modular basada en escenas.
- Uso correcto de herencia en Godot.
- Manejo de señales para desacoplar lógica.
- Control de estados globales.
- Manejo de pausa y process_mode.
- Resolución de bugs complejos relacionados con colisiones.
- Creación de assets, frames, personajes, enemigos, escenarios, fondos, botones personalizados.
- Creación de animaciones por frames.
- Edición y creación de imágenes para los assets.